UGDSAI 29 · PROBLEM 9 · GROUP 9

SupportEscalator

A LangGraph support-ticket agent that resolves routine SaaS tickets automatically and pauses for a human supervisor when policy, sentiment, or unresolved work demands escalation.

Human-in-the-Loop | Conditional Routing | Pydantic v2 | Streamlit | 30/30 tests

Team

Divyam Jindal · Bhanu Uday · Ishaan Bansal · Harman Manik

Format: PDF deck · 8 min presentation + 2 min Q&A

SEGMENT 1 · PROBLEM FRAMING

Mosaic Cloud needs faster support without unsafe automation.

2,000+
TICKETS/DAY
Volume from the brief

40%
CURRENT UNASSISTED
Starting point

75%
AUTO-RESOLUTION TARGET
Business goal

$200+
REFUND APPROVAL
Supervisor threshold

> The business risk is not just slow support. It is resolving risky tickets without the right human review, or sending every routine ticket to a human and wasting capacity.

SEGMENT 1 · USER + KPIS

The target user is the support operations lead.

What they need

- Routine tickets resolved in seconds
- Risky cases escalated with context
- Clear audit trail for refunds and angry customers
- Evidence that automation is behaving correctly

KPIs we track

- Auto-resolution rate: target 75%
- Mean time to resolution
- CSAT for frustrated customers
- Escalation appropriateness
- Refund exposure routed to supervisors

SEGMENT 2 · SOLUTION OVERVIEW

One graph. Four specialist paths. One escalation gate.

01
Ticket input
customer message + account id

02
Classifier
bug / billing / feature / general

03
Sentiment
frustration score feeds risk

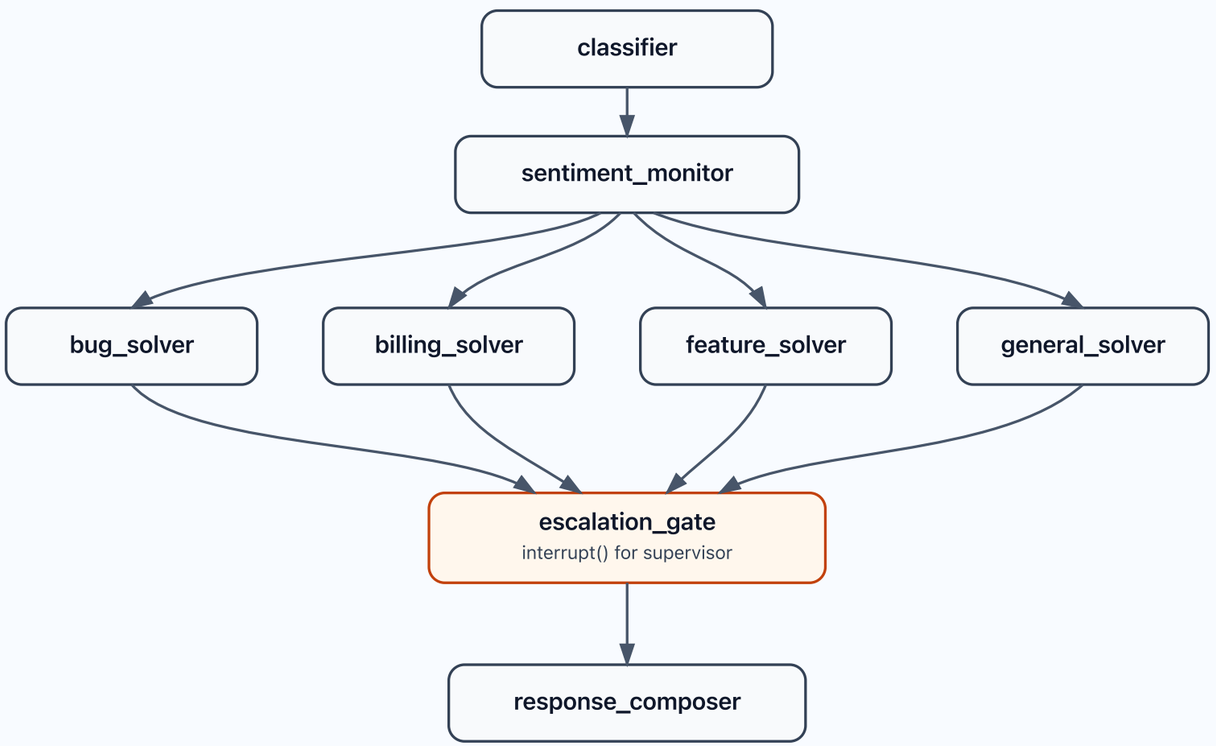
04
Solver
KB search, account lookup, refund calc

05
Escalation gate
interrupt if angry, refund high, unresolved

06
Final response
customer reply + metadata

This is intentionally not a single chatbot. The control flow is explicit, inspectable, and resumable.

Architecture diagram committed in the repository.



Source: `assets/support_escalator_graph.svg`

Why this pattern scores

- Conditional edges route by category.
- `interrupt()` creates real human-in-the-loop escalation.
- `SqliteSaver` persists paused runs.
- Pydantic state makes every transition inspectable.

SEGMENT 2 · TYPED STATE

The state schema is the product contract.

FIELD	PURPOSE
`ticket`	Original customer issue
`category`	Literal route: bug, billing, feature, general
`resolution_attempts`	Audit trail of solver outputs
`sentiment_score`	Risk signal into escalation
`escalation_reason`	Human review trigger
`supervisor_input`	Resume payload from the human
`final_response`	Customer-facing answer

Engineering proof

- OpenAI structured output for classifier + sentiment.
- Deterministic fallback when no API key is set.
- UI normalization prevents Pydantic/dict shape crashes.
- Checkpoint tests verify pause/resume behavior.

30 / 30 tests

6 evaluation cases

SEGMENT 3 · DEMO FLOWS

The demo proves all required behaviours.

1. Password reset

General ticket resolves automatically.

Route	`general_solver`
Escalation	None
Outcome	Final response ready

2. Duplicate billing

Refund exceeds the supervisor threshold.

Route	`billing_solver`
Trigger	$499 refund > $200
Outcome	`interrupt()` + resume

3. Angry upload bug

Tone and unresolved bug both create risk.

Route	`bug_solver`
Trigger	sentiment ≥ 0.67
Outcome	Supervisor guidance included

SEGMENT 4 · EVALUATION AND RESULTS

We tested the graph like an evaluator would.

30/30
TESTS PASS
latest local run

6
EVAL SCENARIOS
all expected behaviours

4
SOLVER ROUTES
bug, billing, feature, general

3
ESCALATION TRIGGERS
sentiment, refund, unresolved

REPRESENTATIVE CASE	EXPECTED BEHAVIOUR	PROOF
Password reset	general route, no interrupt	`test_general_ticket_resolves_without_interrupt`
Duplicate billing	billing route, refund interrupt, resume	`test_billing_refund_interrupts_and_resumes`
Angry upload bug	bug route, sentiment interrupt	`test_angry_bug_ticket_interrupts_for_sentiment`
Unknown account	unresolved billing escalates	`tests/test_evaluation.py`

SEGMENT 4 · BUSINESS DASHBOARD

The Streamlit UI is an operations console, not just a form.

Customer / agent view

- Ticket intake and demo queue
- Customer/account snapshot
- Resolution timeline and solver evidence
- Final response copy/download

Supervisor / manager view

- Pending escalation reason
- Refund exposure and account context
- Approve/reject with guidance
- Analytics: auto-resolution, escalations, sentiment, category mix

SEGMENT 4 · BONUS COVERAGE

Bonus work is real, but scoped honestly.

Implemented / documented

- LangSmith tracing: env-driven observability setup.
- Evaluation framework: 6 named representative cases.
- Prompt iteration: classifier and sentiment prompts documented in README.
- Supervisor summary: ticket, sentiment, category, reason, attempted resolution.

Next production step

- SLA timers and breach-triggered escalation.
- Zendesk sandbox adapter for real ticket ingest.
- Multilingual response drafting.
- Postgres checkpointer and supervisor assignment queue.

SEGMENT 4 · RISKS AND LEARNINGS

The important fixes are now part of the system.

What broke

- `ResolutionAttempt` was a Pydantic object, not a dict.
- Memory-only state made restart demos fragile.
- External graph rendering was unreliable in sandboxed network conditions.

How we fixed it

- Added UI state normalization and regression tests.
- Switched default graph persistence to `SqliteSaver`.
- Committed a stable SVG diagram artifact.

SEGMENT 5 · CLOSE + SEGMENT 6 · Q&A

Thank you.

The panel should remember: explicit LangGraph routing, real human pause/resume, 30 passing tests, and a credible support-ops story. Repository, deck, architecture diagram, business memo, and demo script are committed for submission.

github.com/DivyamJindal/deploying-ai-agents-exam | 30 / 30 tests | LangGraph + Streamlit | SqliteSaver checkpointing

Prepared Q&A themes

- Why this LangGraph pattern instead of one chatbot?
- What happens when a node fails or a supervisor rejects?
- What minimum data is required to make this useful?
- How would this scale beyond the demo?